

Royalty Money Engine — worker spec

The highest-risk logic in the platform. A bug here pays artists the wrong amount, which is trust-death for a distributor. Read this before writing the import/payout workers.

Non-negotiable principles

1. Integer money only. All amounts are minor units (USD cents, ETB santim). No floats, ever.
2. Accrue in report currency (USD). Convert to ETB only at payout, with the rate recorded on the payout row. Never convert inside the ledger.
3. Idempotent imports. Re-running the same report must never double-credit. Enforced two ways: report-level guard + a unique index on the ledger.
4. Reserve on payout. The ledger is debited when a payout is created, not when it succeeds. This prevents double-spend under concurrency. On failure, reverse with a credit.

5. Every money movement is a ledger entry.
Balance is always derived, never stored.
-

Two schema additions (apply before building)

- (1) Idempotency backstop: one ledger entry per source object.

```
create unique index up_ledger_source_unique
on ledger_entries(source_type, source_id)
where source_id is not null;
```

- (2) Report-level dedupe: don't import the same statement twice.

```
create unique index up_royalty_report_natural
on royalty_reports(source, source_ref, period_start,
period_end);
```

Correction to the schema comment on `payouts` :
debit the ledger on payout
creation (reserve), not on success. See §5.

Pipeline

```
IMPORT → MATCH → ALLOCATE → LEDGER CREDIT
(one txn per report)
```

```
PAYOUT: RESERVE (debit) → EXECUTE → PAID |  
FAILED → REVERSE (credit)
```

Import/match/allocate/credit run inside one DB transaction per report —
all or nothing. Payout is its own transaction.

1. Import (idempotent)

```
begin  
  if exists report(source, source_ref, period_start,  
period_end) with status='processed':  
    return -- already done, no-op  
  upsert report row, status='processing'  
  bulk insert royalty_lines (row,  
match_status='unmatched')  
  ... (match + allocate below) ...  
  set report.status='processed', gross_minor, line_count  
  commit -- on any error: rollback → report stays re-  
runnable
```

2. Match

For each unmatched line:

- Normalize ISRC: uppercase, strip
hyphens/spaces.
- `matched_track_id := track where normalize(isrc) =
normalize(line.isrc)` .

- Found → `match_status='matched'` . Not found → stays `unmatched` → manual queue.
- MVP matches on ISRC only. UPC+sequence fallback is phase 2.

Unmatched lines carry no allocations until resolved.

Resolving one later runs

§3–§4 for that single line in its own transaction.

3. Allocate (largest-remainder — no santim lost)

Given a matched line with revenue `R` (integer minor units, may be negative

for clawbacks) and the track's recording splits `p[l..n]` (`numeric(7,4)`,

`sum = 100`):

```

raw_i    = R * p_i / 100          -- rational
floor_i  = trunc(raw_i)           -- toward zero
allocated = sum(floor_i)
remainder = R - allocated         -- 0..±(n-1) minor units
-- distribute the remainder one minor unit at a time
order splits by descending fractional part frac_i = raw_i -
floor_i
(ties broken by track_split.id ascending — deterministic)
for k in 1..abs(remainder):
  floor_[k] += sign(remainder)    -- +1 (or -1 for
clawbacks)

```

```
share_i = floor_i
assert sum(share_i) == R          -- must hold exactly
```

Each `share_i` → one `royalty_allocations` row (may be negative).

Worked example — $R = 1000\text{¢}$, splits A 33.3333, B 33.3333, C 33.3334:

- raw: $333.333 / 333.333 / 333.34 \rightarrow$ floors 333 / 333 / 333 = 999, remainder 1
- largest frac = C (.34) → C += 1 → 333 / 333 / 334 = 1000. No cent lost.

Tie example — $R = 1001\text{¢}$, splits A 50 / B 50:

- floors 500 / 500 = 1000, remainder 1, equal fracs → lower id wins → A 501 / B 500.

4. Ledger credit

For each allocation:

- `amount >= 0` → `ledger_entries(entry_type='credit', amount_minor=amount, source_type='royalty', source_id=allocation.id, currency=line.currency)`
- `amount < 0` (clawback) → `entry_type='debit', amount_minor=abs(amount)`, same source.

The unique index on `(source_type, source_id)` makes a replay a no-op even if

the report guard is bypassed. Balance stays correct.

5. Payout (FX applied here)

```
begin
  lock payee row (SELECT ... FOR UPDATE)
  avail = available_balance(payee, 'USD')    -- see §6
  assert requested_usd <= avail
  insert payout(status='requested', source_currency='USD',
               source_amount_minor=requested_usd)
  -- RESERVE immediately:
  insert ledger_entries(entry_type='debit',
                       amount_minor=requested_usd,
                       currency='USD', source_type='payout',
                       source_id=payout.id)
  commit

  -- execute against provider (Telebirr/Chapa) outside the
  txn:
  rate    = fx_source(USD → ETB)            -- your chosen
  source
  amount_etb = round_half_up(requested_usd * rate) --
  integer santim
  call provider(amount_etb, payout_method)

  on success: payout.status='paid', fx_rate=rate,
  amount_minor=amount_etb,
               currency='ETB', provider_ref=..., paid_at=now()
  on failure: payout.status='failed', failure_reason=...
  -- REVERSE the reservation:
  insert ledger_entries(entry_type='credit',
```

```
amount_minor=source_amount_minor,  
            currency='USD', source_type='refund',  
            source_id=payout.id)
```

Because the reversal uses `source_id = payout.id` with `source_type='refund'` (distinct from the `'payout'` debit), both rows coexist under the unique index.

6. Available balance, hold period, clawbacks

Raw balance is `v_payee_balance`. Available (payable) balance is stricter:

```
available(payee, ccy) =  
    credits from allocations whose report.period_end +  
    hold_days <= today - matured  
    - debits from payouts (reservations + paid)
```

- `hold_days` (default 45, in `app_settings`) delays paying out very recent earnings, because DSP statements get revised. This is your clawback buffer.
- Negative lines / clawbacks: a revised report can carry negative revenue → negative allocations → ledger debits. A payee's balance can legitimately go

negative; carry it and offset against future earnings. Block payouts while $\text{balance} \leq 0$.

- Never let a payout push available balance below zero (the `FOR UPDATE` lock + assertion in §5 guarantees this under concurrency).

7. Failure & retry semantics

Stage	Fails mid-way	Recovery
Import/allocate	txn rolls back	report stays <code>processing</code> /re-runnable; unique indexes prevent dupes on retry
Payout reserve	txn rolls back	no reservation, no payout row
Provider call	payout <code>failed</code> + reversal credit	funds returned to balance; artist can retry
Provider timeout (unknown)	do not auto-	mark <code>processing</code> , reconcile against

Stage	Fails mid-way	Recovery
	reverse	provider_ref before deciding

The last row matters: on an ambiguous provider timeout, never blindly reverse — reconcile first, or you risk double-paying.